

Министерство образования Республики Беларусь

Учреждение образования

**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ
И РАДИОЭЛЕКТРОНИКИ**

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина Архитектура вычислительных систем

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему

АВТОМАТИЧЕСКИЙ ДОЗАТОР ЖИДКОСТИ НА БАЗЕ ARDUINO

БГУИР КП 6-05 0612 02 018 ПЗ

Студент

Д. А. Левшуков

Руководитель

А. А. Калиновская

Нормоконтролёр

А. А. Калиновская

Минск 2025

СОДЕРЖАНИЕ

Введение.....	5
1 Архитектура вычислительной системы.....	6
1.1 Структура и архитектура вычислительной системы.....	6
1.2 История, версии и достоинства	7
1.3 Обоснование выбора вычислительной системы.....	8
1.4 Анализ выбранной вычислительной системы.....	9
2 Платформа программного обеспечения.....	10
2.1 Структура и архитектура платформы	10
2.2 История, версии и достоинства	11
2.3 Обоснование выбора платформы	12
2.4 Анализ программного обеспечения для написания программы	12
3 Теоретическое обоснование разработки программного продукта.....	14
3.1 Обоснование необходимости разработки.....	14
3.2 Используемые технологии программирования	14
3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением.....	16
4 Проектирование функциональных возможностей программы	18
5 Архитектура разрабатываемой программы.....	21
5.1 Общая структура программы.....	21
5.2 Описание функциональной схемы программы.....	22
5.3 Описание блок-схемы алгоритма программы.....	23
5.4 Тестирование	24
Заключение	30
Список использованных источников	31
Приложение А (обязательное) Справка о проверке на заимствование.....	33
Приложение Б (обязательное) Листинг программного кода.....	34
Приложение В (обязательное) Функциональная схема алгоритма, реализующего программное средство	41
Приложение Г (обязательное) Блок схема алгоритма, реализующего программное средство	42
Приложение Д (обязательное) Диаграмма вариантов использования	43
Приложение Е (обязательное) Схема алгоритма работы устройства.....	44
Приложение Ж (обязательное) Ведомость курсового проекта	45

ВВЕДЕНИЕ

Повсеместное распространение автоматических дозаторов для жидкого мыла, антисептиков и других моющих средств стало стандартом в общественных местах, офисах и медицинских учреждениях. Их ключевые преимущества — гигиеничность (исключается прямой контакт с поверхностью дозатора) и экономия расходных материалов. Это не только повышает общий уровень санитарной безопасности, но и способствует формированию культуры чистоты, побуждая людей соблюдать гигиену чаще и охотнее. Автоматизация процесса делает его максимально удобным и быстрым, что особенно важно в местах с высоким трафиком людей.

Актуальность данного курсового проекта заключается в практическом изучении принципов проектирования микроконтроллерных систем на примере конкретного и востребованного устройства. Разработка дозатора на базе микроконтроллера Arduino Nano является сравнительно недорогим решением, при этом его функционал легко модифицировать под конкретные задачи. OLED-дисплей SSD1306 помогает сделать устройство более удобным при эксплуатации, а также играет важную роль в отображении текущего состояния системы и сигнализации об ошибках.

Целью курсового проекта является разработка, сборка и программная реализация функционального прототипа бесконтактного дозатора жидкостей на базе микроконтроллера Arduino Nano, использующего два ультразвуковых датчика для надёжного обнаружения руки, дисплейную индикацию на OLED-экране и управление насосом через релейный модуль. Устройство должно автоматически выдавать калибруемую заданную порцию жидкости, обеспечивая оперативную индикацию основных состояний и возможных ошибок при ограниченных ресурсах микроконтроллера.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 Изучить принципы работы и ключевые компоненты систем автоматической дозировки.
- 2 Изучить компоненты и особенности их совместной работы.
- 3 Спроектировать будущую систему.
- 4 Написать и отладить код для Arduino Nano.
- 5 Собрать и протестировать систему.
- 6 Составить схему алгоритма работы устройства и программы.

Данный курсовой проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 100% (отчёт о проверке на заимствования прилагается).

1 АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ

1.1 Структура и архитектура вычислительной системы

Архитектура вычислительной системы для проекта автоматического дозатора на базе Arduino Nano основана на гарвардской архитектуре с разделением памяти программ и данных. Центральным ядром системы является микроконтроллер ATmega328P, который содержит 32 КБ флэш-памяти для хранения кода, 2 КБ оперативной памяти SRAM для работы с переменными и 1 КБ EEPROM для долгосрочного хранения данных. Тактовая частота процессора составляет 16 МГц, что обеспечивает достаточное быстродействие для задач опроса датчиков и управления исполнительными механизмами в реальном времени. Платформа предоставляет цифровые и аналоговые порты ввода-вывода, а также последовательные интерфейсы, необходимые для подключения периферии [1].

Плата Arduino Nano представлена на рисунке 1.1.

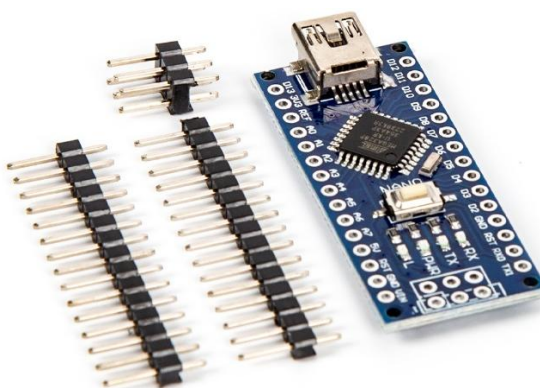


Рисунок 1.1 — Плата Arduino Nano

Arduino Nano получает питание через USB разъем или от внешнего источника напряжения 6-20 В, подаваемого на вывод VIN. Рекомендуемый диапазон питания 7-12 В обеспечивает стабильную работу без перегрева линейного стабилизатора напряжения. Питание 5 В можно подавать напрямую на соответствующий вывод, но это требует использования стабилизированного источника.

Плата содержит 22 цифровых вывода ввода-вывода, из которых 6 могут работать как ШИМ-выходы, и 8 аналоговых входов с разрешением 10 бит. Для работы с аналоговыми датчиками предусмотрен опорный напряжения AREF [2]. Микроконтроллер ATmega328P поддерживает последовательную коммуникацию через аппаратный UART, а также интерфейсы I2C (Two-Wire

Interface) и SPI, реализуемые программно.

Программирование осуществляется через USB с использованием загрузчика Bootloader, что позволяет загружать код без внешних программаторов. Среда разработки Arduino IDE предоставляет упрощенный интерфейс для написания и отладки программ на языке C++ [3]. Особенностью Nano является компактное исполнение с расположением выводов в два ряда для монтажа на макетную плату, что делает ее удобной для прототипирования устройств автоматизации.

1.2 История, версии и достоинства

Платформа Arduino Nano была разработана как компактный аналог Arduino Uno с сохранением основных функциональных возможностей. Первые версии Nano выпускались компанией Gravitech с 2008 года и быстро завоевали популярность благодаря малому размеру (18×45 мм) и возможности монтажа на макетную плату. Исходная версия использовала микроконтроллер ATmega168 с 16 КБ флэш-памяти, но с 2010 года перешла на ATmega328P с 32 КБ памяти, что стало стандартом для текущих ревизий.

Ключевыми достоинствами Arduino Nano являются ее минимальные габариты при полном сохранении 14 цифровых и 8 аналоговых выводов, наличие стабилизатора напряжения 5В позволяющего работать от широкого диапазона питающих напряжений (7-12В), а также встроенный USB-интерфейс через микросхему FTDI или CH340 для программирования. Отличительной особенностью Nano является расположение выводов в два ряда с шагом 2,54 мм, оптимизированное для установки на макетные платы.

По сравнению с Uno, Nano не имеет стандартного разъема питания и использует мини-USB (в ранних версиях) или микро-USB (в современных ревизиях), что делает плату более компактной. При этом сохраняется полная программная совместимость с Uno, включая работу с периферией через стандартные библиотеки Arduino IDE [4].

Современные версии Nano выпускаются различными производителями с дополнительными улучшениями, такими как использование более дешевого USB-контроллера CH340 вместо FT232, что снижает стоимость платы. Некоторые производители предлагают версии с другими микроконтроллерами (например, Nano Every на ATmega4809), но классическая Nano на ATmega328P остается самой популярной для образовательных проектов.

1.3 Обоснование выбора вычислительной системы

Выбор Arduino Nano в качестве вычислительной платформы для проекта автоматического дозатора жидкости обусловлен конкретными техническими требованиями и практическими соображениями. Основные функции системы включают опрос двух ультразвуковых датчиков HC-SR04, управление погружным насосом через одноканальный релейный модуль 5 В, а также вывод диагностической и служебной информации на OLED-дисплей SSD1306 по интерфейсу I²C. Эти задачи не требуют высокой вычислительной мощности, но критически важны с точки зрения надёжности, предсказуемого времени отклика и простоты реализации алгоритмов реального времени.

Arduino Nano оптимально соответствует указанным требованиям благодаря своей архитектуре и компоновке. Микроконтроллер ATmega328P с тактовой частотой 16 МГц и 2 КБ оперативной памяти обеспечивает достаточную производительность для обработки данных с двух датчиков расстояния, обслуживания интерфейса на дисплее и управления релейным модулем. Наличие 14 цифровых выводов и отдельных линий шины I²C позволяет подключить все необходимые компоненты: два датчика HC-SR04 (4 вывода), OLED-дисплей по I²C (2 вывода) и вход релейного модуля (1 вывод), при этом сохраняется резерв по свободным пинам для возможного дальнейшего расширения функциональности системы.

Ключевым преимуществом Arduino Nano перед другими платформами является ее компактность и возможность монтажа непосредственно на макетную плату, что особенно важно для создания компактного корпуса дозатора. По сравнению с Arduino Uno, Nano имеет аналогичные характеристики, но существенно меньшие габариты (18×45 мм), что делает ее предпочтительным выбором для встраиваемых систем.

Важным фактором выбора стала развитая экосистема платформы Arduino, включающая подробную документацию, готовые библиотеки для работы с датчиками и дисплеями, а также активное сообщество пользователей. Это значительно ускоряет процесс разработки и отладки системы. Объем Flash-памяти 32 КБ полностью достаточен для размещения программы управления дозатором с учетом использования стандартных библиотек.

Экономический аспект также сыграл роль в выборе: стоимость Arduino Nano и совместимых компонентов остается низкой, что важно для учебного проекта. При этом платформа обеспечивает необходимое качество и надежность работы, подтвержденные многочисленными практическими применениями в аналогичных проектах автоматизации.

Таким образом, выбор Arduino Nano в качестве вычислительной

системы для автоматического дозатора жидкости является технически и экономически обоснованным, поскольку платформа полностью соответствует требованиям проекта по производительности, количеству портов ввода-вывода, габаритам и простоте программирования.

1.4 Анализ выбранной вычислительной системы

Микроконтроллер ATmega328P, используемый в Arduino Nano, представляет собой 8-битное устройство с RISC-архитектурой, оптимизированной для выполнения большинства инструкций за один такт при тактовой частоте 16 МГц. Технические характеристики включают 32 КБ флэш-памяти для хранения программы, 2 КБ статической оперативной памяти SRAM для данных и 1 КБ EEPROM для сохранения параметров. Микроконтроллер имеет 23 программируемых линии ввода-вывода, поддерживает периферийные интерфейсы USART, SPI и I2C.

Для проекта автоматического дозатора жидкости вычислительных ресурсов ATmega328P достаточно для реализации всех необходимых функций. Программный код, включающий алгоритм опроса двух ультразвуковых датчиков, управление помпой, работу с OLED-дисплеем SSD1306 по интерфейсу I2C и светодиодной индикацией, занимает примерно 5-7 КБ флэш-памяти (20-25% от общего объема). Объем оперативной памяти 2 КБ хватает для размещения переменных и буферов данных с запасом около 50%.

Архитектура микроконтроллера обеспечивает эффективную работу с периферийными устройствами благодаря встроенным подсистемам прерываний, позволяющим быстро реагировать на сигналы от датчиков. Поддержка интерфейса I2C обеспечивает простоту подключения OLED-дисплея SSD1306 без значительной нагрузки на процессорное ядро.

Важным преимуществом для проекта является низкое энергопотребление микроконтроллера, что позволяет использовать компактные источники питания. Наличие EEPROM памяти позволяет сохранять настройки устройства, такие как длительность подачи жидкости, даже после отключения питания.

Анализ показывает, что ATmega328P полностью соответствует требованиям проекта по производительности, объему памяти и периферийным возможностям, обеспечивая надежное выполнение алгоритма управления дозатором.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Структура и архитектура платформы

Программное обеспечение Arduino IDE представляет собой интегрированную среду разработки, специально созданную для программирования микроконтроллеров платформы Arduino. Основой разработки является язык C++ с упрощённым набором функций и библиотек, адаптированных для начинающих пользователей. Ключевой особенностью программной архитектуры является концепция скетчей, состоящих из двух обязательных функций: `setup()` для однократной инициализации параметров и `loop()` для циклического выполнения основной программы [5].

Процесс компиляции и загрузки программ осуществляется через цепочку инструментов `avr-gcc`, включающую препроцессор, компилятор и линкер. Важной особенностью Arduino является автоматическая генерация прототипов функций и подключение стандартных заголовочных файлов, что значительно упрощает процесс разработки. Загрузка прошивки происходит через `bootloader`, записанный в память микроконтроллера, что позволяет программировать устройство через стандартный USB-интерфейс без использования внешних программаторов.

Библиотечная система Arduino включает несколько категорий компонентов: стандартные библиотеки (например, `Wire`, `SPI`, `SoftwareSerial`) для работы с базовыми интерфейсами, сторонние библиотеки для конкретных датчиков и модулей, а также пользовательские библиотеки [6]. Для данного проекта наиболее актуальными являются библиотеки для работы с ультразвуковыми датчиками расстояния, с OLED-дисплеем на контроллере SSD1306 по интерфейсу I2C, а также библиотеки для управления исполнительными устройствами через релейный модуль.

Взаимодействие с аппаратной частью реализовано через систему функций аппаратной абстракции: `pinMode()` для настройки режима работы выводов, `digitalWrite()` и `digitalRead()` для работы с цифровыми сигналами, `analogRead()` для чтения аналоговых значений, а также `delay()` и `millis()` для управления временными интервалами. Отладка программ осуществляется через последовательный порт с использованием функций `Serial.begin()` и `Serial.print()`, что позволяет выводить отладочную информацию на компьютер.

Для данного проекта программное обеспечение будет включать инициализацию периферийных устройств в функции `setup()` (двух ультразвуковых датчиков HC-SR04, OLED-дисплея SSD1306 и релейного модуля насоса), циклический опрос датчиков расстояния в функции `loop()`,

алгоритм управления насосом с точным таймингом подачи жидкости, визуализацию текущего состояния системы на OLED-дисплее и обработку исключительных ситуаций (отсутствие объекта, слишком малое/большое расстояние). Код разработан с учётом модульности и возможности последующего расширения функциональности, например изменения алгоритма дозирования или добавления новых режимов индикации.

2.2 История, версии и достоинства

История развития Arduino IDE тесно связана с эволюцией аппаратной платформы Arduino. Первая версия среды разработки была выпущена в 2005 году и базировалась на открытой платформе Processing, изначально созданной для визуализации данных. Основной целью разработчиков было создание простой и доступной среды для программирования микроконтроллеров, которая бы не требовала глубоких знаний в электронике [7].

Со временем Arduino IDE прошла значительный путь развития. Ранние версии (линейка 1.0.x) предлагали базовый функционал для написания и загрузки кода, тогда как современные версии (2.x) получили более совершенный редактор с автодополнением кода, встроенным менеджером библиотек и плат, а также рядом функций, характерных для современных интегрированных сред разработки. Важным этапом стало появление Arduino IDE 1.5 с поддержкой новых плат на базе 32-битных микроконтроллеров и расширенной системой описания аппаратных платформ.

Ключевыми достоинствами Arduino IDE являются её кроссплатформенность (поддержка Windows, macOS, Linux), простота установки и настройки, а также интеграция с менеджером плат, позволяющим легко добавлять поддержку новых устройств. Особенно важна для образовательных проектов встроенная поддержка последовательного монитора для отладки программ, что упрощает анализ работы микроконтроллерных систем в реальном времени.

Несмотря на некоторые ограничения (относительно простой функционал отладки по сравнению с профессиональными средами), Arduino IDE остаётся оптимальным выбором для учебных и учебно-практических проектов благодаря сочетанию простоты использования и достаточного набора возможностей для реализации таких устройств, как автоматический дозатор жидкости.

2.3 Обоснование выбора платформы

Выбор Arduino IDE в качестве основной среды разработки для проекта обусловлен совокупностью практических фактов.

Проект ориентирован на микроконтроллер ATmega328P, для которого Arduino IDE является стандартным и максимально оптимизированным инструментом. В отличие от альтернативных сред разработки, он не требует сложной первоначальной настройки и позволяет перейти к написанию и загрузке кода буквально после установки программы [8].

Для реализации поставленных задач — опроса двух ультразвуковых датчиков, управления погружным насосом через релейный модуль и вывода статуса работы на OLED-дисплей SSD1306 по интерфейсу I2C — достаточно базового функционала Arduino IDE. Все необходимые библиотеки для работы с перечисленными компонентами (библиотеки для датчиков расстояния, дисплея SSD1306 и вспомогательные библиотеки для работы с шиной I2C) могут быть установлены через встроенный менеджер библиотек, что существенно ускоряет процесс разработки и упрощает сопровождение проекта.

Наконец, кроссплатформенность Arduino IDE, которая работает под Windows, macOS и Linux, обеспечивает универсальность и удобство работы. Это позволяет без труда продолжить разработку на другом компьютере, если возникнет такая необходимость [9].

Таким образом, Arduino IDE полностью удовлетворяет требованиям проекта по критериям простоты освоения, достаточности функционала и доступности, что делает ее оптимальным инструментом для успешного выполнения курсового проекта.

2.4 Анализ программного обеспечения для написания программы

Для реализации системы автоматического дозатора жидкости используется набор специализированных библиотек Arduino, обеспечивающих взаимодействие с аппаратными компонентами и упрощающих разработку прикладного кода.

Библиотека Wire.h предоставляет функции для работы с интерфейсом I2C/TWI, который используется в проекте для связи микроконтроллера с OLED-дисплеем на контроллере SSD1306 [10]. Для самого дисплея применяется библиотека Adafruit_SSD1306 в сочетании с графической библиотекой Adafruit_GFX, обеспечивающими инициализацию экрана, вывод текста и простых графических элементов [11]. Это позволяет обойтись без

реализации низкоуровневого протокола обмена с дисплеем и сосредоточиться на логике работы дозатора.

Для работы с ультразвуковыми датчиками расстояния HC-SR04 в проекте используется стандартный функционал Arduino без дополнительных специализированных библиотек. Измерение расстояния реализовано напрямую через формирование короткого управляющего импульса на выводе запуска датчика и последующее измерение длительности ответного сигнала на выводе приёма с помощью функции `pulseIn()`, после чего время пролёта ультразвука переводится в расстояние по известной скорости звука. Такой подход позволяет точно контролировать таймауты ожидания эха и самостоятельно задавать допустимый диапазон измеряемых расстояний.

При взаимодействии с аппаратной частью используются базовые функции платформы Arduino: `pinMode()` для настройки режимов работы выводов, `digitalWrite()` для управления входом релейного модуля насоса и формированием триггерных импульсов датчиков, а также `delay()` и `millis()` для реализации коротких пауз и отсчёта интервала работы насоса. Архитектура программы построена по модульному принципу: отдельные функции отвечают за измерение расстояния, оценку исправности датчиков, принятие решения о включении насоса, управление релейным модулем и обновление информации на дисплее, что повышает читаемость и удобство сопровождения кода.

Таким образом, используемый набор библиотек Arduino (Wire, Adafruit_SSD1306/Adafruit_GFX и базовые функции ядра Arduino) образует удобную и достаточно мощную программную платформу, полностью соответствующую требованиям курсового проекта автоматического дозатора жидкости по функциональным возможностям, простоте реализации, сопровождаемости и дальнейшей модернизации программного кода.

Открытый характер используемых библиотек и их широкая поддержка со стороны сообщества разработчиков обеспечивают хорошую документированность и наличие готовых примеров кода, что дополнительно снижает трудозатраты на разработку и отладку. При необходимости программная часть может быть расширена за счёт подключения новых датчиков или модулей, причём существующая архитектура кода и выбранный стек библиотек позволяют вносить такие изменения без серьёзной переработки всей системы, сохраняя совместимость и повышая надёжность функционирования дозатора в изменяющихся условиях эксплуатации.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Обоснование необходимости разработки

Контактное управление дозаторами жидкостей повышает риск перекрёстного загрязнения и снижает дисциплину соблюдения гигиенических процедур. В учреждениях здравоохранения именно бесконтактные системы считаются предпочтительными, поскольку уменьшают вероятность передачи патогенов и способствуют более строгому соблюдению правил обработки рук; подобные подходы отражены, в частности, в руководствах ВОЗ по гигиене рук в медицинских организациях и подтверждаются современными обзорами, посвящёнными автоматизированным системам контроля и мониторинга гигиены рук [12].

Для практического применения важна не только бесконтактность, но и воспроизводимость объёма дозы. Ряд исследований, посвящённых коммерческим бесконтактным дозаторам, показывает, что фактический объём подаваемой жидкости может заметно отличаться от заданного, что обуславливает необходимость калибровки и программного контроля дозы на уровне алгоритмов. Разработка собственного программного обеспечения позволяет увязать воедино время работы насоса, учёт пусковых задержек и влияние вязкости жидкости, формируя единую модель, обеспечивающую стабильный объём выдачи.

Таким образом, создание специализированного программного обеспечения для рассматриваемой аппаратной платформы является необходимым условием достижения трёх основных целей: реализации бесконтактного взаимодействия с пользователем в соответствии с современными требованиями гигиены, обеспечения стабильной и настраиваемой дозы при изменении характеристик насоса и свойств жидкости, а также формирования открытого и модифицируемого решения с развёрнутой системой индикации и диагностики. В совокупности это делает проект экономически целесообразным и адаптивным к дальнейшей модернизации.

3.2 Используемые технологии программирования

Программное обеспечение дозатора реализовано на языке C++ в среде Arduino, что обеспечивает прямую компиляцию в машинный код микроконтроллера и предсказуемое поведение в режиме реального времени. Архитектурно проект опирается на классическую структуру скетча с

функциями `setup()` и `loop()`: в первой выполняется инициализация периферии и параметров, во второй — непрерывный цикл обработки датчиков, логики дозирования и индикации. Такой подход соответствует рекомендуемой модели программирования платформы Arduino и облегчает организацию реактивного приложения.

Для управления временем и кооперативной «многозадачности» используется счётчик миллисекунд `millis()`, позволяющий запускать периодические процедуры без блокировки основного цикла; функция `delay()` применяется только там, где допустима кратковременная пауза, чтобы не нарушать отзывчивость системы. Подобное использование `millis()` как неблокирующего таймера рекомендуется в руководствах и практических примерах по Arduino-разработке [13]. Благодаря этому удаётся одновременно опрашивать два ультразвуковых датчика, обновлять индикацию на дисплее и точно обрабатывать импульсы включения насоса.

Связь с OLED-дисплеем 0,96" на контроллере SSD1306 реализована по шине I²C через стандартную библиотеку Wire. Выбор интерфейса I²C минимизирует число задействованных выводов и упрощает разводку, а использование готовой библиотеки для SSD1306 (Adafruit_SSD1306) ускоряет разработку экранного интерфейса и даёт доступ к готовым примитивам отрисовки текста и графики.

Алгоритмы обработки данных ультразвуковых датчиков HC-SR04 построены на измерении длительности импульса на выводе Echo с помощью функции `pulseIn()` и последующем переводе времени пролёта звука в расстояние, что соответствует типовым примерам использования HC-SR04 с Arduino [14]. Низкоуровневое управление выводами контроллера использует стандартные функции `pinMode()` и `digitalWrite()` для конфигурации линий триггера/эха датчиков и силового ключа насоса, что соответствует каноническим практикам Arduino и облегчает переносимость кода.

В совокупности выбранные технологии — событийная петля `setup()/loop()` с неблокирующим таймингом на основе `millis()`, I²C-взаимодействие с OLED через Wire и библиотеку Adafruit_SSD1306, а также измерение времени импульса Echo с конвертацией в расстояние для датчика HC-SR04 — образуют целостную и эффективную программную архитектуру, адекватную требованиям бесконтактного дозирования и удобной индикации на компактной платформе Arduino Nano.

3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением

Аппаратная платформа на базе Arduino Nano с микроконтроллером ATmega328P определяет ключевые архитектурные решения в программной части дозатора. Ограниченный объем оперативной памяти 2 КБ при 32 КБ Flash и 1 КБ EEPROM требует минимизации буферов и строк, хранения неизменяемых данных в программной памяти и отказа от динамических выделений. Под эти ограничения подстраиваются индикация и алгоритмы обработки: программный код спроектирован так, чтобы при одновременном опросе датчиков и управлении насосом укладываться в доступную SRAM, а энергонезависимая память при необходимости может быть использована для хранения коэффициентов калибровки и настроек триггера, не теряющихся при отключении питания. Такое решение соответствует особенностям Гарвардской архитектуры ATmega328P с отдельными областями Flash, SRAM и EEPROM и оставляет задел для дальнейшего развития программного обеспечения.

Модуль OLED 0,96" на контроллере SSD1306 подключён по интерфейсу I²C и при полном кадровом буфере 128×64 бит требует 1024 байта ОЗУ, то есть около половины доступной SRAM. Это накладывает ограничение на индикацию: в программном обеспечении используется текстовый интерфейс с минимальным количеством графики и продуманным режимом обновления экрана без лишней перерисовки. Наличие аппаратного модуля TWI (I²C) в ATmega328P позволяет реализовать обмен с дисплеем через стандартную библиотеку Wire, опираясь на аппаратную поддержку протокола и не перегружая ядро микроконтроллера лишними программными операциями.

Два ультразвуковых датчика HC-SR04 напрямую задают структуру алгоритмов опроса. Каждый датчик требует короткого триггерного импульса и формирует ответ Echo, длительность которого пропорциональна расстоянию до объекта; между измерениями необходимо выдерживать паузу не менее 60 мс во избежание переотражений и взаимных помех. В программной архитектуре это отражено в чередовании запусков левого и правого датчиков по таймерам: триггерные импульсы формируются с безопасным разносом по времени, а вычисленное расстояние дополнительно проверяется на принадлежность допустимому диапазону. Для измерения применяются функции pulseIn() и millis(), что позволяет параллельно обслуживать индикацию и управление насосом, не вводя длительных блокирующих задержек и сохраняя предсказуемое время реакции.

Реализация «мягкого» реального времени опирается на классическую

для Arduino архитектуру `setup()/loop()` и использование счётчика `millis()` вместо продолжительных вызовов `delay()`. Основная программа рассматривается как набор периодических задач: опрос ультразвуковых датчиков, обновление OLED-дисплея, контроль длительности включения насоса и периодическая самодиагностика. Для каждой задачи определён собственный период, а запуск реализуется сравнением текущего значения `millis()` с моментом последнего вызова. Такой подход обеспечивает одновременное обновление индикации, регулярный опрос датчиков и точный учёт времени работы насоса, от которого зависит объём выдаваемой порции. Неблокирующая схема выбрана, поскольку на базовой платформе отсутствует RTOS, а использование длительных `delay()` приводило бы к пропуску измерений и визуальному «подвисанию» интерфейса.

Исполнительный механизм — погружной насос постоянного тока — подключён к системе через одноканальный релейный модуль 5 В / 10 А, а не непосредственно к выводу микроконтроллера. Релейный модуль содержит встроенный драйвер, защитный диод и согласующие цепи, поэтому Arduino формирует лишь логический уровень на входе IN, а вся силовая коммутация выполняется контактами электромеханического реле. Это позволяет безопасно коммутировать токи насоса, превышающие допустимую нагрузочную способность выводов ATmega328P, и гальванически развязать низковольтную логику от силовой части. В программном обеспечении это выражается в управлении одним цифровым пином по типовой схеме, рекомендуемой для проектов на базе Arduino [16]: логика формирования импульса включения, защита от ложных и повторных срабатываний реализуются на уровне таймеров и анализа состояния датчиков, тогда как детали силовой коммутации остаются внутри стандартного модуля реле.

Таким образом, структура и поведение программы разрабатывались в прямой привязке к ограничениям и возможностям аппаратной платформы: объём SRAM определяет формат и способ обновления OLED-интерфейса, аппаратное I²C (TWI) задаёт модель обмена с дисплеем, акустические особенности HC-SR04 формируют расписание опроса, а наличие релейного модуля — схему управления насосом. Полученная взаимосвязь архитектуры вычислительной системы и программного обеспечения обеспечивает стабильную дозировку, предсказуемое время реакции и безопасную работу дозатора в реальных условиях эксплуатации. В данном подразделе показано, что программная реализация неотделима от выбранной элементной базы: все ключевые алгоритмы — от опроса датчиков до формирования индикации и управления насосом — опираются на реальные ресурсы микроконтроллера Arduino Nano и учитывают его архитектурные ограничения.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

Программное обеспечение бесконтактного дозатора жидкости разработано как прикладная система реального времени для платформы Arduino Nano на основе микроконтроллера ATmega328P. Архитектура программы построена по модульному принципу: выделены блоки инициализации, измерения расстояния двумя ультразвуковыми датчиками HC-SR04, принятия решения о подаче жидкости, управления насосом через одноканальный релейный модуль и отображения состояния на OLED-дисплее SSD1306. Такое разбиение упрощает сопровождение кода, локализацию ошибок и дальнейшее расширение функциональности. Основная задача программного обеспечения — обнаруживать наличие рук в зоне дозатора, запускать насос на фиксированное время при соблюдении заданных условий, отслеживать работоспособность датчиков и информировать пользователя о текущем состоянии устройства через экран. При этом программа работает в условиях ограниченных ресурсов ATmega328P, что требует неблокирующей организации времени на основе счётчика `millis()` и экономного использования оперативной памяти.

При включении питания выполняется инициализация всех периферийных линий и библиотек. В функции `setup()` настраиваются направления работы выводов ультразвуковых датчиков (пары линий запуска и приёма для левого и правого HC-SR04), цифровой выход управления релейным модулем насоса и встроенный светодиод контроллера. Затем инициализируется шина I²C и библиотека работы с OLED-дисплеем, после чего дисплей очищается и на экран выводится стартовое сообщение о запуске системы и переходе в диагностический режим. Отдельная процедура самотеста выводит на экран результаты проверки линий, а последующая оценка состояния датчиков выполняется уже в основном цикле программы по результатам регулярных измерений расстояния: если в течение ряда попыток по каналу приходят только некорректные или выходящие за допустимый диапазон значения, датчик считается неисправным и система блокирует работу насоса до восстановления нормальных измерений.

Измерение расстояния до объекта реализовано в функции `readDistance()`. Для каждого HC-SR04 формируется стандартный триггерный импульс: кратковременный высокий уровень на выводе `trig`, после чего с помощью функции `pulseIn()` измеряется длительность высокого уровня на входе `Echo`. В случае успешного измерения длительность переводится в расстояние по стандартной формуле с учётом скорости звука; при отсутствии эха в течение

заданного таймаута функция возвращает специальное значение, обозначающее ошибку измерения. В основном цикле `loop()` функция `readDistance()` вызывается для обоих датчиков последовательно. На основе полученных расстояний формируются логические признаки `hand1` и `hand2`, отражающие наличие рук в зоне срабатывания: значение признака становится истинным, если расстояние положительно и не превышает порог `maxDistance`.

Логика работы насоса реализована в основном цикле как простой конечный автомат с двумя ключевыми состояниями: «насос выключен» и «насос включен». В состоянии покоя программа анализирует признаки `hand1` и `hand2`. Подача разрешается только при одновременном обнаружении рук обоими датчиками; при выполнении этого условия релейный модуль активируется (вывод `relayPin` переводится в нужное состояние), переменная `pumpOn` становится истинной, а в `pumpStartTime` фиксируется момент начала дозирования по значению `millis()`. На дисплей выводится соответствующее сообщение о запуске насоса на заданное время. В состоянии активной подачи контролируются два критерия завершения: уход рук из зоны (оба признака `hand1` и `hand2` становятся ложными) и превышение максимально допустимой длительности работы насоса `PUMP_DURATION`. В первом случае подача прерывается раньше, реле отключается и на экране отображается информация об отмене активации; во втором — цикл дозирования штатно завершается по таймеру, насос выключается, а на дисплей выводится сообщение о завершении импульса. Во всех случаях используется неблокирующий контроль времени на основе `millis()`, что обеспечивает воспроизводимость объёма порции без блокировки основного цикла. В случае одновременного отказа обоих датчиков включение насоса блокируется, при активной подаче насос принудительно отключается, а на дисплей выводится аварийное сообщение, защищающее систему от неконтролируемой подачи жидкости при потере информации о положении рук.

Интерфейс пользователя реализован на OLED-дисплее `SSD1306` с использованием двух вспомогательных функций: `showLines()` и `showStatus()`. Функция `showLines()` принимает до четырёх строк текста, очищает экран, устанавливает курсор в начало и последовательно выводит переданные строки, после чего отправляет буфер на дисплей. Она используется для отображения стартовых, диагностических и аварийных сообщений (результат самотеста, предупреждения о проблемах с датчиками, статусы ошибок). Функция `showStatus()` формирует стандартный экран рабочего режима: текущее расстояние от каждого датчика (`S1`, `S2`) с указанием их активности и состояние насоса. Она вызывается периодически с заданным интервалом, обеспечивая пользователю наглядное представление о текущем состоянии

системы. За счёт текстового формата и минимальной графики достигается экономное использование кадрового буфера контроллера SSD1306 и оперативной памяти микроконтроллера. Дополнительно в цикле реализовано мигание встроенным светодиодом, служащее простым индикатором активности контроллера.

Организация времени в программе основана на неблокирующей схеме с использованием счётчика `millis()` вместо продолжительных вызовов `delay()`. Основной цикл рассматривается как набор периодических задач: опрос датчиков расстояния, обновление индикации, проверка состояния датчиков, контроль длительности работы насоса. Для каждой задачи определён собственный период, а запуск реализуется сравнением текущего значения `millis()` с моментом последнего вызова соответствующей функции. Это позволяет одновременно поддерживать интерфейс на дисплее, регулярно опрашивать ультразвуковые модули и точно отрабатывать временные интервалы дозирования. Неблокирующая схема необходима, поскольку на базовой платформе отсутствует операционная система реального времени, а использование длительных `delay()` приводило бы к пропуску измерений и визуальным «подвисаниям» интерфейса.

Архитектура программного обеспечения дозатора ориентирована на дальнейшее развитие и расширение набора функций без радикальной переработки базового кода. На уровне измерений возможно добавление более сложных алгоритмов фильтрации расстояния, а также алгоритмов обнаружения нестабильной работы датчиков с автоматическим переходом на одноканальный режим при отказе одного из HC-SR04. В части дозирования система может быть дополнена механизмами калибровки объёма порции: реализацией сервисного режима, в котором длительность включения насоса подбирается экспериментально и сохраняется в энергонезависимой памяти, а также введением нескольких профилей дозирования с выбором пользователем.

В результате спроектированный набор функциональных возможностей — неблокирующий основной цикл с кооперативным таймингом, обработка двух ультразвуковых каналов с учётом их физических ограничений, управление насосом через релейный модуль с контролем длительности импульса и информативная индикация на OLED — напрямую следует из выбранной аппаратной платформы и требований к гигиеничной, воспроизводимой и безопасной выдаче жидкости. Заложенная модульная архитектура программы обеспечивает возможность дальнейшего расширения функциональности без пересмотра базовых принципов работы системы.

5 АРХИТЕКТУРА РАЗРАБАТЫВАЕМОЙ ПРОГРАММЫ

5.1 Общая структура программы

Программное обеспечение бесконтактного дозатора реализовано как модульная система с единым неблокирующим циклом `loop()`, в котором кооперативно выполняются все основные задачи: опрос двух ультразвуковых датчиков HC-SR04, вычисление признаков наличия рук в зоне дозатора, управление погружным насосом через одноканальный релейный модуль 5 В / 10 А и обновление индикации на OLED-дисплее SSD1306. Входной точкой является функция `setup()`, где выполняется настройка линий ввода-вывода для датчиков и реле, инициализация интерфейса I²C и библиотеки `Adafruit_SSD1306`, а также вывод стартового экрана и запуск встроенной процедуры самодиагностики `testSensorPins()`. После завершения инициализации управление полностью переходит в функцию `loop()`, которая непрерывно выполняется до отключения питания.

Внутри основного цикла выделяются несколько логических подсистем. Подсистема сбора данных реализована через функцию измерения расстояния, которая формирует триггерные импульсы на выводах запуска обоих HC-SR04, измеряет длительность ответного сигнала на линиях приёма с помощью таймера и переводит её в расстояние, при этом значения вне допустимого диапазона трактуются как отсутствие валидного эха. На основе последовательности последних измерений для каждого датчика отдельно оценивается его работоспособность: если в течение ряда циклов фиксируются только корректные расстояния, датчик считается исправным, при накоплении последовательности ошибочных измерений он помечается как неисправный. Подача жидкости допускается только при исправной работе обоих каналов; при обнаружении отказа хотя бы одного датчика насос немедленно отключается, а на дисплей выводится экран с сообщением об ошибке и указанием проблемного канала. Подсистема принятия решения опирается на результаты измерений и признаков наличия рук в зоне каждого датчика, логика дозирования разрешает включение насоса только при одновременной активации обоих каналов. Подсистема управления насосом реализована через учёт текущего состояния и момента включения: в зависимости от прошедшего времени насос либо продолжает работу, либо отключается по истечении заданного интервала или при уходе рук из зоны. Подсистема индикации, построенная на текстовых экранах, отображает служебные сообщения, ошибки, текущие расстояния и статус насоса, а также при необходимости выводит напоминание о желательности перезапуска питания после ранее

произошедшего отказа. Дополнительно в цикле реализовано мигание встроенного светодиода с периодом порядка одной секунды, выступающее простым индикатором активности микроконтроллера.

5.2 Описание функциональной схемы программы

Функциональная схема программы, выполненная в соответствии с ГОСТ 2.701–2008 [17], отображает общую структуру и взаимосвязи между её основными компонентами. Центральным управляющим элементом является микроконтроллер, который координирует работу всех подсистем. К нему подключены два измерительных ультразвуковых датчика: они определяют расстояние до объектов в зоне контроля и передают данные на обработку. Система питания обеспечивает стабильное напряжение для контроллера, датчиков и исполнительных устройств.

Функционально система организована в виде пяти взаимодействующих блоков. Блок запуска и инициализации выполняет первичную настройку аппаратных модулей после подачи питания: конфигурирует порты, инициализирует интерфейс I²C для дисплея, сбрасывает таймеры и устанавливает начальные состояния, после чего выводит стартовое сообщение на экран. Блок анализа значений с датчиков обрабатывает поступающие от них данные: выполняет фильтрацию, проверяет корректность измерений, анализирует статистику для выявления сбоев и формирует логические сигналы о наличии объектов в зоне контроля и исправности самих датчиков. На основе этих сигналов блок синхронизации работы программы и насоса реализует логику управления дозированием: определяет момент для начала подачи жидкости при одновременном обнаружении объектов в зоне обоих датчиков, отслеживает заданный интервал работы, обрабатывает команды на досрочное прерывание и инициирует аварийное отключение при неисправности датчиков. Сформированная этим блоком команда передаётся в блок управления работой насоса, который преобразует логический сигнал в физическое воздействие на релейный модуль, коммутирующий питание электронасоса, тем самым осуществляя непосредственную подачу жидкости. Блок управления работой дисплея обеспечивает интерфейс взаимодействия с пользователем. По интерфейсу I²C он выводит на OLED-экран различную информацию: текущие расстояния с датчиков, статус работы насоса, сообщения о процессе дозирования, предупреждения об ошибках. Все блоки работают в едином цикле, синхронизированном внутренними таймерами контроллера, образуя замкнутую систему «измерение – анализ – действие – отображение», обеспечивая автоматическую и безопасную работу дозатора.

5.3 Описание блок-схемы алгоритма программы

Блок-схема алгоритма программы, выполненная в соответствии с ГОСТ 19.701-90 (ИСО 5807-85) [18] и ГОСТ 2.701-2008, отражает последовательность основных этапов работы программного обеспечения от момента подачи питания до выполнения циклов дозирования. В верхней части схемы располагается стартовый блок «Пуск», из которого управление переходит к блоку «Инициализация системы». На этом этапе задаются режимы работы линий ввода-вывода для обоих ультразвуковых датчиков, выхода управления релейным модулем насоса и встроенного светодиода, инициализируется интерфейс PC и модуль работы с OLED-дисплеем, очищается экран и выводится стартовое сообщение о запуске. Затем выполняется блок самодиагностики линий, где проверяются сигналы на тестовых выводах и на экран выдаётся краткий отчёт о результате.

Далее управление переходит к основному циклу, который на блок-схеме изображён в виде замкнутого контура. Первый шаг цикла — обновление сервисных таймеров и состояний: по счётчику миллисекунд определяется необходимость инверсии встроенного светодиода, что служит индикатором активности контроллера. Затем выполняется пара последовательных блоков «Измерение расстояния датчиком 1» и «Измерение расстояния датчиком 2», где для каждого ультразвукового модуля формируется короткий импульс запуска и измеряется время прихода эха с последующим переводом в расстояние, либо фиксируется отсутствие валидного сигнала.

На основе полученных расстояний выполняется блок «Оценка исправности датчиков». В нём для каждого канала анализируется статистика последних измерений: если подряд накоплено достаточное число корректных расстояний, датчик считается исправным, если же в течение ряда циклов приходят только ошибочные либо выходящие за допустимый диапазон значения, датчик помечается неисправным. Далее следует ромб «Оба датчика исправны?». При ответе «нет» управление переходит в блок «Отключить насос, вывести сообщение об ошибке датчиков», где насос гарантированно выключается, на дисплее отображается экран с текстом о неисправности одного или обоих датчиков и указанием измеренных значений, после чего по стрелке основного цикла управление возвращается к очередному измерению расстояний.

Если оба датчика признаны исправными, по ветви «да» выполняется блок «Определение наличия рук в зоне датчиков», где по расстояниям формируются логические признаки для каждого канала в зависимости от сравнения с заданным порогом срабатывания. Затем следует ромб «Насос

сейчас включён?», определяющий ветвление алгоритма. При ответе «нет» выполняется ромб «Обе руки находятся в зоне контроля?». Если условие не выполняется, насос остаётся выключенным и управление переходит к обновлению экрана статуса. Если же оба датчика одновременно фиксируют объект в рабочей зоне, выполняется блок «Включить насос и запомнить момент начала дозирования»: на релейный модуль подаётся управляющий сигнал, запоминается время запуска, на дисплей выводится краткое сообщение о начале подачи порции.

При ответе «да» в ромбе «Насос сейчас включён?» алгоритм переходит в ветку сопровождения активного режима. Сначала проверяется условие «Руки отсутствуют в зоне действия датчиков?». При положительном ответе выполняется блок «Прервать подачу и вывести сообщение об отмене активации», в котором насос немедленно отключается до истечения заданного интервала, а на дисплее кратковременно отображается сообщение о досрочном завершении дозирования. Если руки остаются в зоне контроля, выполняется ромб «Истёк интервал работы насоса?». При достижении заданного времени включения выполняется блок «Отключить насос после истечения времени», где подача жидкости завершается штатно и на экран выводится сообщение о завершении импульса. Если ни одно из условий отключения не наступило, насос продолжает работу до следующего прохода цикла.

Завершающим шагом каждой итерации основного цикла является блок «Обновление экрана статуса», где формируется рабочий экран с заголовком режима, текущими расстояниями по обоим датчикам и состоянием насоса. Если ранее в процессе работы фиксировался отказ датчиков, на этом же экране дополнительно выводится текстовое напоминание о необходимости перезапуска питания для обеспечения корректной работы в дальнейшем. Далее по стрелке цикла управление возвращается к блоку измерения расстояний, обеспечивая непрерывный мониторинг обстановки и автоматическую подачу жидкости при соблюдении всех условий. Описанная блок-схема отражает реальную последовательность действий программы и реализует требуемое поведение дозатора в нормальных и аварийных режимах.

5.4 Тестирование

Тест 1. Нормальный режим работы при исправных датчиках. Целью первого теста является проверка базовой работоспособности системы при корректном подключении всех компонентов. Перед началом испытания оба ультразвуковых датчика HC-SR04 подключены согласно схеме, релейный

модуль и насос запитаны от общего источника 5 В, OLED-дисплей подключён по шине I²C. После включения питания на дисплее должны последовательно отобразиться стартовый экран, затем сообщения самотеста пинов D5–D6 и краткие результаты проверки. Далее устройство переходит в рабочий режим, на экране появляется экран статуса с заголовком RUN MODE, значениями расстояний S1, S2 в сантиметрах и состоянием насоса Pump: OFF. При отсутствии рук в зоне датчиков значения расстояния должны быть стабильными и превышать порог срабатывания, а насос оставаться выключенным. Фактическое отображение этих экранов и общий вид устройства в нормальном режиме работы представлены на рисунках 6.1 и 6.2.



Рисунок 6.1 – Индикация на OLED-дисплее при нормальном режиме работы

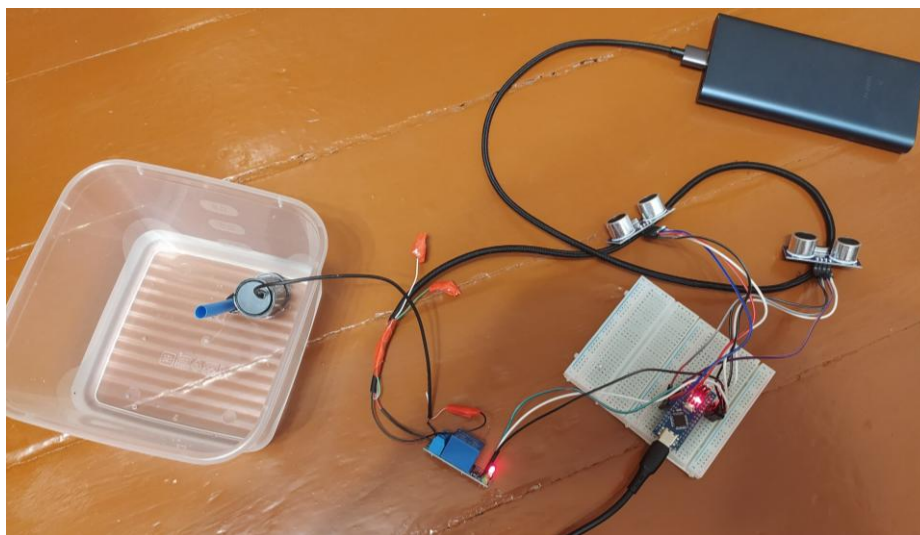


Рисунок 6.2 – Общий вид устройства в нормальном режиме работы

Тест 2. Срабатывание дозатора при наличии обеих рук в зоне датчиков. Во втором тесте проверяется корректность логики включения насоса при одновременном обнаружении рук двумя датчиками. Предусловием является успешное прохождение самотеста и переход системы в режим RUN MODE. Оператор подносит обе руки в рабочую зону таким образом, чтобы

каждая рука находилась в секторе обзора своего датчика HC-SR04 на расстоянии не более 15 см. В этот момент программа должна зафиксировать `hand1 = true` и `hand2 = true`, на дисплее на короткое время появляется экран с текстом `BOTH HANDS / Pump ON / for 2 sec`, слышен щелчок реле и запускается погружной насос. После окончания краткого сообщения система возвращается к экрану `RUN MODE`, при этом в строках `S1` и `S2` указываются уменьшенные значения расстояния, а строка `Pump`: в течение периода подачи показывает состояние `ON`. Визуально фиксируется, что насос выдаёт дозу жидкости, а по истечении примерно двух секунд автоматически отключается. Фактическое отображение показаний дисплея и общий вид устройства в режиме подачи жидкости представлены на рисунках 6.3 и 6.4.



Рисунок 6.3 – Индикация на OLED-дисплее в режиме подачи жидкости

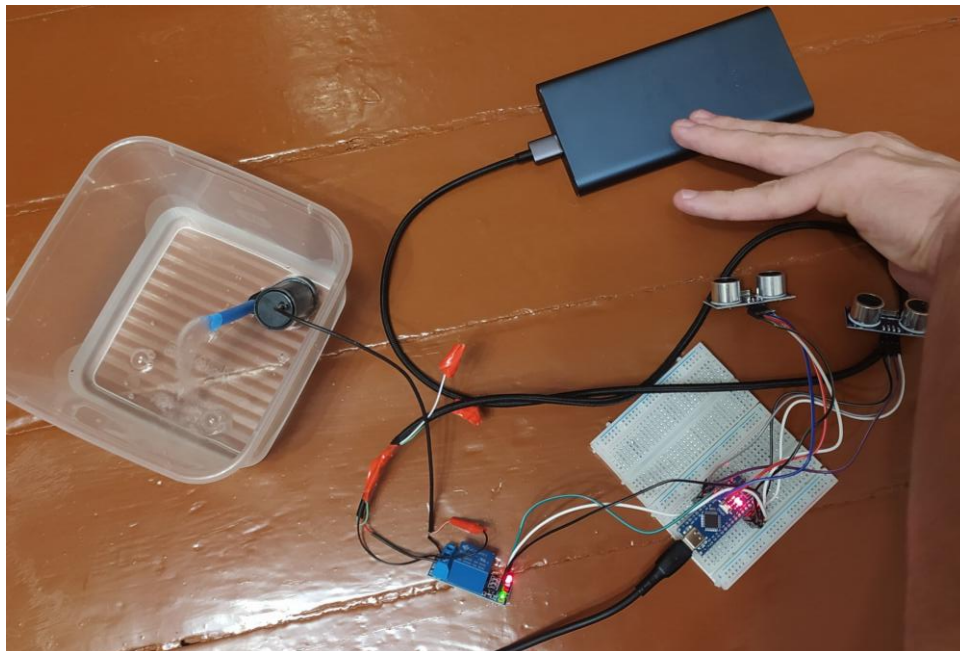


Рисунок 6.4 – Общий вид устройства в режиме подачи жидкости

Тест 3. Преждевременное отключение при отводе рук до завершения дозирования.

Третий тест направлен на проверку реакции системы при уходе рук из зоны

действия датчиков в момент, когда насос уже работает. После перевода системы в рабочий режим оператор подносит обе руки и инициирует подачу, как в тесте 2, а затем целенаправленно убирает руки до истечения интервала в 2 секунды. Ожидается, что при исчезновении признаков hand1 и hand2 программа немедленно переведёт выход реле в состояние отключения, на экран выведет сообщение ACTIVATION / CANCELLED / Pump OFF, и насос прекратит работу. Длительность фактической подачи в этом случае оказывается меньше эталонной, а на последующем экране RUN MODE расстояния S1 и S2 снова увеличиваются, отражая отсутствие объекта в зоне. На рисунках 6.5 представлен дисплей с сообщением об отменённой активации в момент преждевременного отключения.



Рисунок 6.5 – Индикация на OLED-дисплее в момент преждевременного отключения

Тест 4. Отказ одного датчика.

В четвёртом тесте проверяется поведение программы при частичном отказе измерительного тракта. Для имитации неисправности отключается один из ультразвуковых датчиков HC-SR04, при этом второй датчик продолжает работать штатно. После включения питания система проходит стартовый этап и самотест линий, затем в основном цикле по результатам повторяющихся неудачных измерений по одному из каналов алгоритм оценки исправности отмечает этот датчик как неисправный. На дисплее формируется специальный экран с заголовком вида SENSOR ERROR и пояснением, какой именно датчик отсутствует или не выдаёт корректных данных, а также отображаются текущие значения расстояний по обоим каналам. В таком состоянии подача жидкости блокируется: насос принудительно остаётся выключенным независимо от наличия руки в зоне исправного датчика. После восстановления подключения датчика и появления последовательности корректных измерений система возвращается к обычному экрану рабочего режима, однако на нём дополнительно отображается напоминание о том, что ранее фиксировалась ошибка и для полностью корректной работы рекомендуется перезапустить питание устройства. Фактическое отображение показаний дисплея и общий вид устройства с отключённым и затем восстановленным

датчиком представлены на рисунках 6.6 и 6.7.



Рисунок 6.6 – Индикация на OLED-дисплее с отключенным датчиком

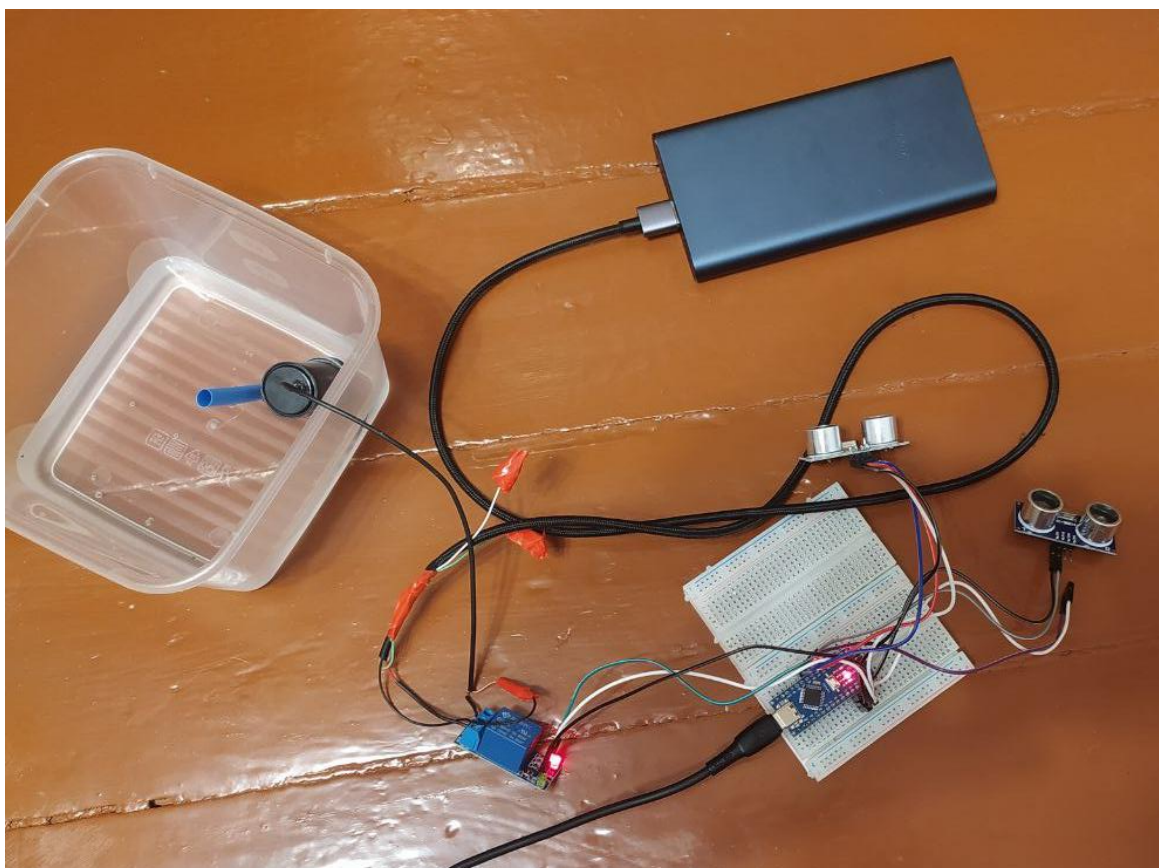


Рисунок 6.7 – Общий вид устройства с отключенным датчиком

Тест 5. Полный отказ датчиков и блокировка насоса.

Пятый тест предназначен для проверки защитного поведения системы при полном отсутствии достоверных измерений расстояния. Для этого имитируется отказ обоих датчиков HC-SR04 путём их отключения от контроллера. После запуска программы самотест линий выполняется штатно, однако в основном цикле все попытки измерения заканчиваются получением некорректных значений, и алгоритм оценки исправности помечает оба канала как неисправные. В этой ситуации формируется экран с сообщением о неисправности датчиков, насос немедленно переводится в выключенное состояние, и дальнейшая подача жидкости блокируется, независимо от действий пользователя. После повторного подключения датчиков и появления

устойчивых валидных измерений система возвращается в рабочий режим; при этом на стандартном экране статуса сохраняется текстовое напоминание о том, что ранее имел место отказ, и для исключения накопленных эффектов рекомендуется выполнить перезапуск питания. Фактическое отображение показаний дисплея и общий вид устройства с отключёнными датчиками представлены на рисунках 6.8 и 6.9.



Рисунок 6.8 – Индикация на OLED-дисплее с отключенными датчиками

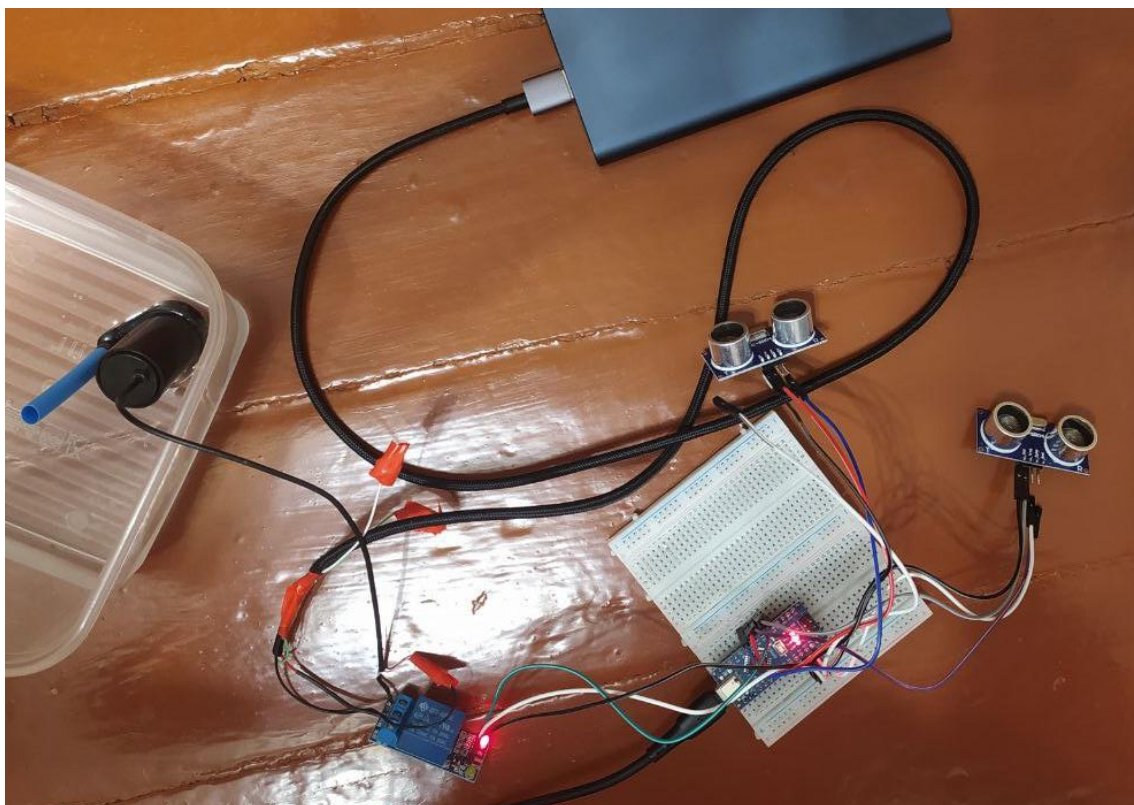


Рисунок 6.9 – Общий вид устройства с отключенным датчиком

По результатам проведённых испытаний можно сделать вывод о работоспособности реализованного алгоритма: система корректно функционирует в нормальном режиме, обеспечивает запуск насоса только при одновременном обнаружении обеих рук, корректно обрабатывает аварийные ситуации, связанные с отказом датчиков.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта был разработан, собран и протестирован прототип бесконтактного дозатора жидкостей на базе микроконтроллера Arduino Nano.

Ключевым результатом работы стала реализация алгоритма дозирования, ориентированного на надёжное обнаружение рук и воспроизводимую подачу объёма за счёт управления насосом по времени. Использование двух независимых датчиков расстояния позволило снизить вероятность ложных срабатываний и повысить устойчивость к случайным объектам в зоне установки. Неблокирующая организация программы на основе архитектуры `setup()/loop()` и счётчика `millis()` обеспечила корректное совмещение задач опроса датчиков, управления исполнительным механизмом и обновления индикации без потери отзывчивости системы.

Проведённое тестирование показало, что разработанное программное обеспечение адекватно реагирует на основные эксплуатационные сценарии: нормальную работу при наличии рук в зоне, досрочное прекращение подачи при их отводе, отказ одного или обоих датчиков. В каждом из случаев система демонстрирует предсказуемое поведение, информирует пользователя о текущем состоянии и предотвращает неконтролируемую подачу жидкости, что особенно важно для задач гигиены и безопасной эксплуатации в бытовых и общественных пространствах.

Полученный прототип может рассматриваться как основа для дальнейшего развития: возможна интеграция более гибких режимов настройки чувствительности и объёма дозы, расширение средств диагностики и индикации, добавление сетевых интерфейсов для учёта срабатываний и удалённого мониторинга.

Результаты курсового проекта подтверждают практическую применимость выбранного подхода к построению бесконтактных дозаторов и демонстрируют, что сочетание простой аппаратной платформы и продуманной программной логики позволяет решить задачу автоматизированной и гигиеничной подачи жидкостей.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Easycraft [Электронный ресурс]. – Режим доступа: <https://easycraft.by/arduino-nano-v3-0-ft232-chip-plus-mini-usb-kabely>. – Дата доступа: 22.09.2025.
- [2] Arduino docs [Электронный ресурс]. – Режим доступа: <https://docs.arduino.cc/hardware/nano/>. – Дата доступа: 22.09.2025.
- [3] Arduino [Электронный ресурс]. – Режим доступа: <https://www.arduino.cc/en/software/>. – Дата доступа: 22.09.2025.
- [4] Arduino store [Электронный ресурс]. – Режим доступа: <https://store.arduino.cc/products/arduino-nano>. – Дата доступа: 22.09.2025.
- [5] Arduino docs [Электронный ресурс]. – Режим доступа: <https://docs.arduino.cc/software/ide-v1/tutorials/arduino-ide-v1-basics/>. – Дата доступа: 22.09.2025.
- [6] Arduino store [Электронный ресурс]. – Режим доступа: <https://docs.arduino.cc/language-reference/>. – Дата доступа: 22.09.2025.
- [7] IEEE Spectrum [Электронный ресурс]. – Режим доступа: <https://spectrum.ieee.org/the-making-of-arduino/>. – Дата доступа: 22.09.2025.
- [8] Tproger [Электронный ресурс]. – Режим доступа: <https://tproger.ru/curriculum/arduino-quick-start>. – Дата доступа: 22.09.2025.
- [9] AlexGyver [Электронный ресурс]. – Режим доступа: <https://alexgyver.ru/arduino-first>. – Дата доступа: 22.09.2025.
- [10] Arduino Docs [Электронный ресурс]. – Режим доступа: <https://docs.arduino.cc/language-reference/en/functions/communication/wire/>. – Дата доступа: 22.09.2025.
- [11] GitHub [Электронный ресурс]. – Режим доступа: https://github.com/adafruit/Adafruit_SSD1306/. – Дата доступа: 22.09.2025.
- [12] ВОЗ [Электронный ресурс]. – Режим доступа: <https://www.who.int/publications/i/item/9789241597906>. – Дата доступа: 01.10.2025.
- [13] Arduino docs [Электронный ресурс]. – Режим доступа: <https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/>. – Дата доступа: 12.10.2025.
- [14] Arduino Project Hub [Электронный ресурс]. – Режим доступа: <https://projecthub.arduino.cc/lucasfernando/ultrasonic-sensor-with-arduino-complete-guide-284faf> – Дата доступа: 12.10.2025.
- [15] Microchip [Электронный ресурс]. – Режим доступа: <https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive->

Microcontrollers-ATmega328P_Datasheet.pdf – Дата доступа: 18.10.2025.

[16] Arduino Docs [Электронный ресурс]. – Режим доступа: <https://docs.arduino.cc/learn/electronics/transistor-motor-control/>. – Дата доступа: 22.10.2025.

[17] ГОСТ 2.701–2008. ЕСКД. [Электронный ресурс]. – Режим доступа: <https://files.stroyinf.ru/Data2/1/4293830/4293830651.pdf>. Дата доступа: 25.10.2025.

[18] ГОСТ 19.701–90 (ИСО 5807–85). Схемы алгоритмов, программ, данных и систем [Электронный ресурс]. – Режим доступа: <https://www.edsd.ru/files/pdf/GOST-19.701-90.-Shemy-algoritmov-programm-dannyh-i-sistem.-Uslovnye-Oboznacheniya-i-pravila-vypolneniya.pdf>. Дата доступа: 25.10.2025.

ПРИЛОЖЕНИЕ А

(обязательное)

Справка о проверке на заимствования



Антиплагиат 2.0, Проверка и повышение
уникальности текста за 2 минуты

antiplagius.ru

Уважаемый пользователь!

Обращаем ваше внимание, что система Антиплагиус отвечает на вопрос, является тот или иной фрагмент текста заимствованным или нет. Ответ на вопрос, является ли заимствованный фрагмент именно плагиатом, а не законной цитатой, система оставляет на ваше усмотрение.

Отчет о проверке № 9654316

Дата выгрузки: 2025-11-29 00:00:58
Пользователь: dmitrylev2006@gmail.com, ID: 9654316

Отчет предоставлен сервисом «Антиплагиат»
на сайте antiplagius.ru/

Информация о документе

№ документа: 9654316
Имя исходного файла: Проверка текста 2025_10_28 21:00:24.txt
Размер файла: 0.03 МБ
Размер текста: 16841
Слов в тексте: 1983
Число предложений: 85

Информация об отчете

Дата: 2025-11-29 00:00:58 - Последний готовый отчет
Оценка оригинальности: 100%
Заимствования: 0%

100.00%

0%

Источники:

Доля в тексте	Ссылка
---------------	--------

Информация о документе:

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА 3.1 Обоснование необходимости разработки Контактное управление дозаторами жидкостей повышает риск перекрёстного загрязнения и снижает дисциплину соблюдения гигиенических процедур В учреждениях здравоохранения именно бесконтактные системы считаются предпочтительными поскольку уменьшают вероятность передачи патогенов и способствуют более строгому соблюдению правил обработки рук подобные подходы отражены в частности в руководствах ВОЗ по гигиене рук в медицинских организациях и подтверждаются современными обзорами посвящёнными автоматизированным системам контроля и мониторинга гигиены рук 12 Для практического применения важна не только бесконтактность но и воспроизводимость объёма дозы Ряд исследований посвящённых коммерческим бесконтактным дозаторам показывает что фактический объём подаваемой жидкости может заметно отличаться от заданного что обуславливает необходимость калибровки и программного контроля дозы на уровне алгоритмов Разработка собственного программного обеспечения позволяет увязать воедино время работы насоса учёт пусковых задержек и влияние вязкости жидкости формируя единую модель обеспечивающую стабильный объём выдачи Таким образом создание специализированного программного обеспечения для рассматриваемой аппаратной платформы является необходимым условием достижения трёх основных целей реализации бесконтактного взаимодействия с пользователем в соответствии с современными требованиями гигиены обеспечения стабильной и настраиваемой дозы при изменении характеристик насоса и свойств жидкости а также формирования открытого и модифицируемого решения с развёрнутой системой индикации и диагностики В совокупности это делает проект экономически целесообразным и адаптивным к дальнейшей модернизации 3.2 Используемые технологии программирования Программное обеспечение дозатора реализовано на языке C в среде Arduino что обеспечивает прямую компиляцию в машинный код микроконтроллера и

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программного кода

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

const int trigPin1 = 3;
const int echoPin1 = 4;
const int trigPin2 = 5;
const int echoPin2 = 6;
const int relayPin = 2;

const int maxDistance = 15;
const unsigned long PUMP_DURATION = 2000;

bool pumpOn = false;
unsigned long pumpStartTime = 0;

bool sensor1OK = false;
bool sensor2OK = false;

uint8_t sensor1BadCount = 0;
uint8_t sensor2BadCount = 0;
uint8_t sensor1GoodCount = 0;
uint8_t sensor2GoodCount = 0;

bool sensorErrorEver = false;
bool sensorsEverAllOK = false;
```



```

const uint8_t BAD_THRESHOLD = 10;
const uint8_t GOOD_THRESHOLD = 3;

void showLines(const String &l1 = "",
               const String &l2 = "",
               const String &l3 = "",
               const String &l4 = "")
{
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println(l1);
    if (l2.length()) display.println(l2);
    if (l3.length()) display.println(l3);
    if (l4.length()) display.println(l4);
    display.display();
}

void showStatus(long dist1, long dist2, bool pumpOn) {
    display.clearDisplay();
    display.setCursor(0, 0);

    display.println("RUN MODE");

    display.print("S1: ");
    display.print(dist1);
    display.println(" cm");

    display.print("S2: ");
    display.print(dist2);
    display.println(" cm");

    display.print("Pump: ");
    display.println(pumpOn ? "ON" : "OFF");

    if (sensorErrorEver) {
        display.println("Reboot after error");
    }
}

```

```

    }

    display.display();
}

void showSensorError(long dist1, long dist2) {
    String line2;

    if (!sensor1OK && !sensor2OK) {
        line2 = "Sensor 1 & 2 error";
    } else if (!sensor1OK) {
        line2 = "Sensor 1 error";
    } else if (!sensor2OK) {
        line2 = "Sensor 2 error";
    }

    String line3 = "S1:" + String(dist1) + " S2:" + String(dist2);
    showLines("SENSOR ERROR", line2, line3, "Reconnect & reboot");
}

void testSensorPins() {
    showLines("--- D5-D6 TEST ---", "D5 (Trig): HIGH");

    pinMode(5, OUTPUT);
    digitalWrite(5, HIGH);
    delay(100);
    digitalWrite(5, LOW);
    showLines("--- D5-D6 TEST ---", "D5 (Trig): LOW");
    delay(500);

    pinMode(6, INPUT);
    int echoState = digitalRead(6);

    showLines("--- D5-D6 TEST ---",
               "D6 (Echo) state:",
               String(echoState),

```

```

        "--- TEST DONE ---");
    delay(1000);
}

long readDistance(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 20000);
    if (duration == 0) return 999;

    long distance = duration * 0.034 / 2;

    if (distance > 400) return 999;

    return distance;
}

void updateSensorHealth(long dist1, long dist2) {
    bool s1GoodNow = (dist1 != 999 && dist1 > 0 && dist1 < 400);
    bool s2GoodNow = (dist2 != 999 && dist2 > 0 && dist2 < 400);

    if (s1GoodNow) {
        if (sensor1GoodCount < GOOD_THRESHOLD) sensor1GoodCount++;
        sensor1BadCount = 0;
    } else {
        if (sensor1BadCount < BAD_THRESHOLD) sensor1BadCount++;
        sensor1GoodCount = 0;
    }

    if (sensor1GoodCount >= GOOD_THRESHOLD) sensor1OK = true;
    if (sensor1BadCount >= BAD_THRESHOLD) sensor1OK = false;
}

```

```

    if (s2GoodNow) {
        if (sensor2GoodCount < GOOD_THRESHOLD) sensor2GoodCount++;
        sensor2BadCount = 0;
    } else {
        if (sensor2BadCount < BAD_THRESHOLD) sensor2BadCount++;
        sensor2GoodCount = 0;
    }

    if (sensor2GoodCount >= GOOD_THRESHOLD) sensor2OK = true;
    if (sensor2BadCount >= BAD_THRESHOLD) sensor2OK = false;
}

void setup() {
    pinMode(trigPin1, OUTPUT);
    pinMode(echoPin1, INPUT);
    pinMode(trigPin2, OUTPUT);
    pinMode(echoPin2, INPUT);
    pinMode(relayPin, OUTPUT);
    pinMode(LED_BUILTIN, OUTPUT);

    digitalWrite(relayPin, HIGH);
    pumpOn = false;

    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        for (;;) {}
    }
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);

    showLines("=== SYSTEM STARTED ===", "Diag: D5-D6 test");
    delay(1000);

    testSensorPins();
    showLines("WAITING SENSORS", "Connect HC-SR04");
    delay(1000);

```

```

}

void loop() {
    static unsigned long lastBlink = 0;
    if (millis() - lastBlink > 1000) {
        digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
        lastBlink = millis();
    }

    long dist1 = readDistance(trigPin1, echoPin1);
    long dist2 = readDistance(trigPin2, echoPin2);

    updateSensorHealth(dist1, dist2);

    bool allSensorsOK = sensor1OK && sensor2OK;

    if (allSensorsOK) {
        sensorsEverAllOK = true;
    }

    if (!allSensorsOK) {
        if (sensorsEverAllOK) {
            sensorErrorEver = true;
        }

        if (pumpOn) {
            digitalWrite(relayPin, HIGH);
            pumpOn = false;
        }

        showSensorError(dist1, dist2);
        delay(100);
        return;
    }

    bool hand1 = (dist1 > 0 && dist1 <= maxDistance);
    bool hand2 = (dist2 > 0 && dist2 <= maxDistance);

```

```

if (pumpOn) {
  if (!hand1 && !hand2) {
    digitalWrite(relayPin, HIGH);
    pumpOn = false;
    showLines("ACTIVATION", "CANCELLED", "Pump OFF");
    delay(200);
  }
  else if (millis() - pumpStartTime >= PUMP_DURATION) {
    digitalWrite(relayPin, HIGH);
    pumpOn = false;
    showLines("Pump OFF", "2 seconds passed");
    delay(200);
  }
} else {
  if (hand1 && hand2) {
    digitalWrite(relayPin, LOW);
    pumpOn = true;
    pumpStartTime = millis();
    showLines("BOTH HANDS", "Pump ON", "for 2 sec");
    delay(150);
  }
}

showStatus(dist1, dist2, pumpOn);

delay(20);
}

```

ПРИЛОЖЕНИЕ В
(обязательное)
Функциональная схема алгоритма, реализующего
программное средство

ПРИЛОЖЕНИЕ Г
(обязательное)
Блок схема алгоритма, реализующего программное
средство

ПРИЛОЖЕНИЕ Д
(обязательное)
Диаграмма вариантов использования

ПРИЛОЖЕНИЕ Е
(обязательное)
Схема алгоритма работы устройства

ПРИЛОЖЕНИЕ Ж
(обязательное)
Ведомость курсового проекта